

Embedded Challenge.

Objetivo

Avaliar a capacidade de desenhar arquitetura, orquestrar serviços em containers, comunicar entre containers e entregar uma base sólida (Dockerfiles, docker-compose) com documentação clara.

Deliverable 1 - Arquitetura

Produz um diagrama simples que represente:

- **Container A (C/C++)**: simula um módulo de leitura de **QR Code** por porta serial.
 - Deve receber **comandos**:
 - **init**, configura o QR code, serial baudrate, etc
 - **ping**, check periodico para perceber se o QR code está alive, deve responder com pong
 - **start**, espera por leitura do QR code ate timeout X
 - **stop**, cancela caso esteja em espera
 - Deve responder um objecto JSON. A resposta tem de estar feita de forma a que possa ser acessível ao **container B**. (IPC local, REST/gRPC, Message Brokkers, etc...)
- **Container B (Python)**: serviço de **gateway** que:
 - Conecta a um broker MQTT remoto encriptado com recurso a certificados ( test.mosquitto.org)
 - Subscrive-se a tópicos (ex.: `from_cloud/commands`).
 - Publica eventos MQTT na cloud (ex.: leitura de QR, `from_device/events`).
 - Converte mensagens MQTT para comandos do QR code e tem de ser “enviados” para o container A.

O diagrama deve mostrar fluxos locais, componentes e portas/tópicos principais.

Deliverable 2 - Implementação

Container A - QR Serial (C/C++)

- Programa em C/C++ que connecta com uma porta série física/virtual, responsável por gerar e ler strings de texto dessa mesma porta serie.
- Deve:

- a. Abrir a porta série especificada por variáveis de ambiente (ex.:

```
SERIAL_PORT=/dev/ttyS1 ...).
```

- b. Interpretar “comandos/mensagens” enviados pelo **container B**:

```
1 INIT
2 PING
3 START
4 STOP
```

- c. Responder via a mesma forma:

- **START** → fica em espera uma leitura ate dar timeout passado X tempo

```
1 {"qr-data":{"code":"ABC123","ts":1730780000}
2 OU
3 {"qr-data":{"code":"TIMEOUT","ts":1730780000}
```

- **STOP** → pára as leituras e responde com **OK**
- **PING** → responde **PONG**
- **INIT** → responde **OK** se for bem configurado

- d. Permitir configurar parâmetros (ex: device path, velocidade baud) por **variáveis de ambiente**.

- e. Fazer log em stdout e/ou volumes com timestamps.

- f. Expor “**comunicações**” para ser possivel o **container B** usar.

- g. Deve lidar com reabertura da porta série em caso de erro.

- Pode **simular a porta série** do QR code usando **socat** :

```
1 socat -d -d pty,raw,echo=0 pty,raw,echo=0
```

Isso gera duas portas virtuais conectadas do genero loopback e os dados podem ser mockados ou enviados para uma delas.

Container B - Gateway (Python)

- Serviço em Python que:
 - a. Conecta-se ao **broker MQTT** remoto.
 - b. **Subscreve** o tópico de comandos vindos da cloud (`from_cloud/command`) e routea estes comandos para o container A de alguma forma.
 - c. **Subscreve** ao tópico de eventos vindo do **container A** (`from_device/events`) e
 - d. **Publica** os eventos para a cloud `from_device/events`

e. Deve lidar com reconexões MQTT em caso de erro, e export erros para

`from_device/events`

Deliverable 3 — Containerização & Orquestração

- Dockerfiles (um para C/C++ e outro para Python)
- O `docker-compose.yml` deve:
 - Simular o QR code como porta serie (`fake-serial`)
 - Arrancar uma app compilada em C
 - Configurar dependências e healthchecks.
 - Volumes necessarios para storage de logs

Exemplo pseudo docker-compose:

```
1 services:
2   fake-serial:
3     image: alpine/socat
4     command: socat -d -d pty,raw,echo=0,link=/tmp/ttyS1 pty,raw,echo=0,link=/tmp/ttyS2
5     volumes:
6       - /tmp:/tmp
7
8   qr-c:
9     image: alpine/###
10    command: ./qr-c
11    environment:
12      SERIAL_PORT: /tmp/ttyS1
13    volumes:
14      - /tmp:/tmp
15    depends_on:
16      - fake-serial
17
18  gateway-py:
19    image: python:slim-####
20    command: python app.py
21    environment:
22      MQTT_HOST: test.mosquitto.org
23      MQTT_PORT: 8883
24      MQTT_CERT: /home/mosquitto.org.crt
25      MQTT_TOPIC_CMD: from_cloud/command
26      MQTT_TOPIC_EVENT: from_device/events
27    volumes:
28      - /tmp:/tmp
29    depends_on:
30      - qr
```

Deliverable 4 — Documentação

README.md com:

1. **Como executar e replicar setup**
2. **Como testar**, podem ser mocks as dependencias de hardware
3. **Exemplos**
4. **Decisões e racional de design**
5. **Melhorias futuras**

Estrutura Sugerida do Repositório

```
1 | .
2 | ├── diagrams/
3 | |   └── arquitetura.mmd
4 | ├── qr-c/
5 | |   ├── src/
6 | |   |   └── main.c
7 | |   └── Dockerfile
8 | ├── gateway-py/
9 | |   ├── app.py
10 | |   ├── requirements.txt
11 | |   └── Dockerfile
12 | ├── docker-compose.yml
13 | └── README.md
14 |
```

Tempo & Dicas

- **Tempo sugerido:** 3 dias
 - **Scope:** é preferível algo **pequeno mas bem acabado** a algo grande e frágil ou inacabado.
 - **Liberdade:** podes ajustar nomes de tópicos/paths/etc...
-